4.3 MongoDB jako warstwa danych: Baza danych, Auth i Vector Search w stacku z Node.js

Moduł 4: Szybkie Wdrażanie (Ship Fast) w oparciu o Angular

Krzysztof Rytlewski · 2026

MongoDB w stacku AI — dlaczego dokumentowa baza pasuje najlepiej?

MongoDB to dokumentowa baza danych NoSQL przechowująca dane w formacie BSON (binarny JSON). Jej elastyczność jest wyjątkową zaletą dla aplikacji AI — dokumenty nie wymagają stałego schematu, co ułatwia iterowanie podczas prototypowania.

Cecha	Korzyść dla aplikacji AI
Schemat elastyczny	Historia czatu, wyniki AI i metadane mają różną strukturę — brak migracji
Zagnieżdżone dokumenty	Wiadomości sesji to naturalnie tablica obiektów w jednym dokumencie
Atlas Vector Search	Wbudowane wyszukiwanie semantyczne bez osobnego serwisu
Mongoose (ODM)	Warstwa walidacji i schematów po stronie Node.js — opcjonalna, ale wygodna

Połączenie z MongoDB w Node.js — wzorzec singleton

Połączenie z bazą tworzymy raz przy starcie aplikacji i wielokrotnie reużywamy — nie otwieramy nowego połączenia przy każdym żądaniu HTTP.

Moduł `db.js` eksportuje dwie funkcje: `connectDB()` — nawiązuje połączenie przy pierwszym wywołaniu lub zwraca istniejące — oraz `getDB()` — zwraca referencję do bazy lub rzuca wyjątek, jeśli połączenie nie zostało zainicjalizowane.

W `server.js` wywołujemy `await connectDB()` przed uruchomieniem `app.listen()`. Zmienna `MONGODB_URI` ze zmiennych środowiskowych zawiera connection string — **nigdy** nie umieszczamy jej bezpośrednio w kodzie źródłowym.

Modelowanie danych — podejście A: zagnieżdżone wiadomości

W podejściu A cała historia czatu jest częścią jednego dokumentu sesji:

```
{
  _id: ObjectId("..."),
  userId: "user_123",
  title: "Pytania o MongoDB",
  createdAt: ISODate("2026-04-16T10:00:00Z"),
  messages: [
    { role: "user",      content: "Czym jest indeks?", ts: ISODate("...") },
    { role: "assistant", content: "Indeks to ...",      ts: ISODate("...") }
  ]
}
```

Zalety: jeden odczyt = cała sesja, prosta implementacja. **Wada:** dokument może przekroczyć limit 16 MB przy bardzo długich sesjach. Dla typowych aplikacji AI (do kilkuset wiadomości) to podejście jest wystarczające.

Modelowanie danych — podejście B: oddzielna kolekcja wiadomości

W podejściu B wiadomości przechowujemy w osobnej kolekcji powiązanej przez `sessionId`:

```
// Kolekcja sessions
{ _id: ObjectId("s1"), userId: "user_123", title: "...", createdAt: ISODate("...") }

// Kolekcja messages
{ _id: ObjectId("m1"), sessionId: ObjectId("s1"), role: "user",      content: "..." }
{ _id: ObjectId("m2"), sessionId: ObjectId("s1"), role: "assistant", content: "..." }
```

Zalety: brak limitu rozmiaru, łatwe stronicowanie wiadomości. **Wada:** dwa zapytania zamiast jednego przy ładowaniu sesji. Podejście B sprawdza się przy aplikacjach z bardzo długimi historiami rozmów.

Autoryzacja z JWT — przepływ i implementacja

JWT (JSON Web Token) to bezstanowy token podpisany kluczem tajnym. Pozwala każdemu użytkownikowi widzieć tylko swoje sesje.

Przepływ: Angular wysyła `POST /api/auth/login` z danymi, backend weryfikuje hasło przez `bcrypt` i zwraca JWT. Angular przechowuje token i dołącza go do każdego żądania w nagłówku `Authorization: Bearer <token>`. Middleware `requireAuth` weryfikuje podpis tokenu i udostępnia `req.userId` kontrolerom.

Ważne zasady: przechowuj `JWT_SECRET` wyłącznie w zmiennych środowiskowych, ustawiaj krótki czas wygaśnięcia (`exp: '1h'`) i używaj `bcrypt` do hashowania haseł — nigdy czystego tekstu w bazie.

Middleware autoryzacji JWT w Express

```
// middleware/auth.js
import jwt from 'jsonwebtoken';

export function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Brak tokenu autoryzacyjnego' });
  }
  const token = authHeader.slice(7);
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = payload.userId;
    next();
  } catch {
    res.status(401).json({ error: 'Nieprawidłowy lub wygasły token' });
  }
}
```

Chrona trasa filtruje dane po `req.userId` — każdy użytkownik widzi wyłącznie swoje sesje.

Indeksy MongoDB — wydajność zapytań

Bez indeksów MongoDB wykonuje pełne skanowanie kolekcji (*collection scan*) przy każdym zapytaniu. Gdy kolekcja rośnie do tysięcy dokumentów, czas odpowiedzi dramatycznie wzrasta.

Typ indeksu	Zastosowanie
Single field	Filtrowanie po `userId`, `sessionId`
Compound	Sortowanie po wielu polach jednocześnie
TTL	Automatyczne wygasanie danych tymczasowych
Text	Pełnotekstowe przeszukiwanie historii czatu
Vector	Semantyczne wyszukiwanie embeddingów (Atlas Vector Search)

Indeks TTL z `expireAfterSeconds: 90 * 24 * 60 * 60` automatycznie usuwa sesje starsze niż 90 dni. Wywołaj `ensureIndexes()` przy starcie aplikacji — MongoDB pomija tworzenie, jeśli indeks już istnieje.

Tworzenie indeksów w Node.js

```
async function ensureIndexes(db) {
  const sessions = db.collection('sessions');
  const messages = db.collection('messages');

  // Filtrowanie sesji użytkownika z sortowaniem
  await sessions.createIndex({ userId: 1, createdAt: -1 });

  // Indeks TTL — automatyczne usuwanie po 90 dniach
  await sessions.createIndex(
    { createdAt: 1 },
    { expireAfterSeconds: 90 * 24 * 60 * 60 }
  );

  // Paginacja wiadomości w sesji
  await messages.createIndex({ sessionId: 1, ts: 1 });
}
```

Indeks `{ userId: 1, createdAt: -1 }` obsługuje zarówno filtrowanie po użytkowniku, jak i sortowanie od najnowszej sesji.

Środowisko deweloperskie — MongoDB w Dockerze

Podczas developmentu uruchamiamy MongoDB lokalnie przez Docker, by nie łączyć się z klastrem Atlas (oszczędność kosztów, brak zależności od sieci).

Do `docker-compose.yml` dodajemy dwa serwisy: `mongo` (obraz `mongo:7`, port 27017, wolumen `mongo_data` do persystencji danych) oraz `mongo-express` (interfejs GUI dostępny na porcie 8081 bez uwierzytelnienia w środowisku dev).

```
docker compose up -d mongo mongo-express
# MongoDB na localhost:27017
# Mongo Express (GUI) na http://localhost:8081
```

Zmienne środowiskowe dla różnych środowisk

Nigdy nie umieszczaj sekretów bezpośrednio w kodzie ani repozytorium. Stosuj pliki `.env` lokalnie i bezpieczne usługi w produkcji.

```
# .env (development — Docker lokalny)
MONGODB_URI=mongodb://localhost:27017
DB_NAME=aiapp
JWT_SECRET=dev-secret-change-in-prod

# .env.production (Atlas — produkcja)
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/
DB_NAME=aiapp_prod
JWT_SECRET=<wartość z AWS Secrets Manager>
```

W produkcji (AWS Lambda) zmienne środowiskowe definiujemy bezpośrednio w konfiguracji funkcji Lambda lub przez AWS Secrets Manager — plik `.env.production` **nigdy** nie trafia do repozytorium GitHub (dodaj go do `.gitignore`).